

GREEN BOX — Complete Codebase Audit

An exhaustive, evidence-based inspection of the current repository and live Supabase project — not a re-read of prior reports. Every claim below traces to a file path, line number, live query result, or build/lint output captured during this audit.

- 4 independent research passes (storefront, admin, backend/DB/security, cross-cutting)

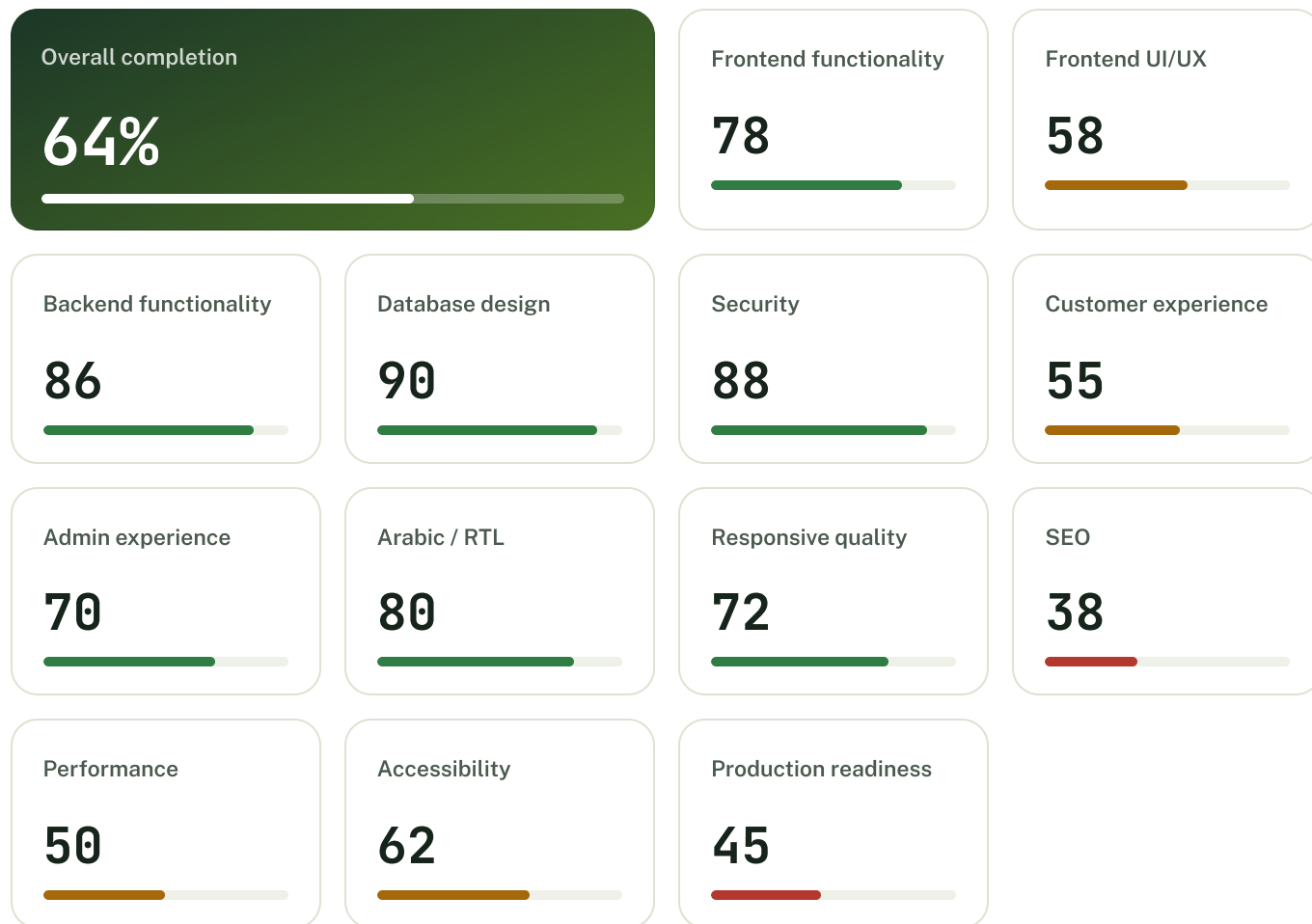
- Live Supabase queries against `pg_policies` , `information_schema` , `pg_cron`

- `npx tsc --noEmit` → **0 errors**
- `npx eslint .` → **0 warnings**

- `npx next build` → **success, 56 routes**

- No browser/visual rendering was performed — visual findings are code-derived only

Scored against the actual, currently-running codebase and live database — not the ambition described in prior reports. Methodology per score is explained in §31.



Every meaningful feature discovered, scored independently across Frontend / Backend / Database / Integration. A feature is only **COMPLETE** when all four actually work together end-to-end — verified by tracing UI → action/RPC → table/policy, not by file existence.

AREA	FEATURE	FRONTEND	BACKEND	DATABASE	INTEGRATION	STATUS
Customer	Homepage	Complete	Complete	Complete	Complete	 Polished
Customer	Category index & listing	Complete	Complete	Complete	Complete	 Complete
Customer	Product detail page	Complete	Complete	Complete	Complete	 Polished
Customer	Search (text)	Complete	Complete	Complete	Complete	 Polished
Customer	Search suggestions	Complete	Complete	Complete	Complete	 Complete
Customer	Add to cart	Complete	Complete	Complete	Complete	 Complete
Customer	Cart view / modify qty / remove	Complete	Complete	Complete	Complete	 Complete
Customer	Cart item notes	Missing	Complete	Complete	None	 Missing
Customer	Checkout — address selection	Complete	Complete	Complete	Complete	 Complete
Customer	Checkout — delivery date	Complete	Complete	Complete	Complete	 Complete
Customer	Checkout — delivery slot + capacity	Complete	Complete	Complete	Complete	 Complete
Customer	Checkout — payment method selection	Complete	Complete	Complete	Complete	 Complete
Customer	Checkout — loyalty redemption	Complete	Complete	Complete	Complete	 Complete
Customer	Checkout — order creation	Complete	Complete	Complete	Complete	 Complete
Customer	Checkout — error messaging	Broken	Complete	N/A	Broken	 Broken
Customer	Order confirmation	Complete	Complete	Complete	Complete	 Complete

AREA	FEATURE	FRONTEND	BACKEND	DATABASE	INTEGRATION	STATUS
Customer	Order list / detail / tracking	Complete	Complete	Complete	Complete	 Complete
Customer	Reorder	Complete	Complete	Complete	Complete	 Polish
Customer	Profile edit	Partial	Complete	Complete	Partial	 Polish
Customer	Address management (CRUD + default)	Complete	Complete	Complete	Complete	 Polish
Customer	Loyalty balance + history (view)	Complete	Complete	Complete	Complete	 Complete
Customer	Loyalty earn / redeem / reverse	Complete	Complete	Complete	Complete	 Complete
Customer	Subscription creation	Complete	Complete	Complete	Complete	 Complete
Customer	Subscription list / pause / resume / cancel	Complete	Complete	Complete	Complete	 Complete
Customer	Subscription detail view	Missing	Complete	Complete	None	 Missing
Customer	Subscription renewal engine	N/A	Complete	Complete	Complete	 Complete
Customer	Green Box bundle browsing	Complete	Complete	Complete	 Blocked	 Blocked
Customer	Login	Complete	Complete	Complete	Complete	 Complete
Customer	Login post-redirect (? next=)	Broken	Broken	N/A	Broken	 Broken
Customer	Register	Complete	Complete	Complete	Complete	 Complete
Customer	Forgot / reset password	Missing	Missing	N/A	Missing	 Missing
Customer	Locale switch (AR↔EN)	Partial	N/A	N/A	Partial	 Polish
Admin	Admin login	Complete	Complete	Complete	Complete	 Complete
Admin	Dashboard / stats	Complete	Complete	Complete	Complete	 Complete
Admin	Staff / role management	Complete	Complete	Complete	Complete	 Complete

AREA	FEATURE	FRONTEND	BACKEND	DATABASE	INTEGRATION	STATUS
Admin	Products CRUD	Complete	Complete	Complete	Complete	Polish
Admin	Product images (upload/delete/primary)	Complete	Complete	Complete	Complete	Polish
Admin	Box contents editor	Complete	Complete	Complete	Complete	Polish
Admin	Categories CRUD	Complete	Complete	Complete	Complete	Polish
Admin	Orders list / detail	Complete	Complete	Complete	Complete	Complete
Admin	Order status control	Complete	Complete	Complete	Complete	Complete
Admin	Payment verification control	Broken	Complete	Complete	Broken	Broken
Admin	Customers list / detail	Complete	Complete	Complete	Complete	Complete
Admin	Delivery zones / areas / slots CRUD	Complete	Complete	Complete	Complete	Complete
Admin	Payment methods management	Partial	Complete	Complete	Partial	Polish
Admin	Loyalty settings	Complete	Complete	Complete	Complete	Complete
Admin	Loyalty manual adjustment	Complete	Complete	Complete	Complete	Complete
Admin	Subscriptions list / detail / status	Complete	Complete	Complete	Complete	Complete
Admin	Manual renewal trigger	Complete	Complete	Complete	Complete	Complete
Admin	CMS — banners	Partial	Complete	Complete	Partial	Polish
Admin	CMS — store settings	Broken	Complete	Complete	Broken	Broken
Platform	RLS security model	N/A	Complete	Complete	Complete	Complete
Platform	Trusted SECURITY DEFINER functions	N/A	Complete	Complete	Complete	Complete

AREA	FEATURE	FRONTEND	BACKEND	DATABASE	INTEGRATION	STATUS
Platform	Storage (media bucket + policies)	N/A	Complete	Complete	Complete	✔ Complete
Platform	Proof-of-payment upload	Missing	Missing	Partial	None	● Missing
Platform	Migration tracking	N/A	Partial	Partial	Partial	● Polished
Platform	Testing infrastructure	Missing	Missing	Missing	Missing	● Missing
Cross-cutting	SEO metadata	Partial	Complete	Complete	Partial	● Polished
Cross-cutting	Sitemap / robots	N/A	Complete	Complete	Partial	● Polished
Cross-cutting	Structured data (JSON-LD)	Partial	N/A	N/A	Partial	● Polished
Cross-cutting	Bilingual / RTL system	Complete	N/A	N/A	Complete	● Polished
Cross-cutting	Glassmorphism design system	Partial	N/A	N/A	Partial	● Polished
Cross-cutting	Placeholder imagery system	Complete	N/A	N/A	Complete	● Polished
Cross-cutting	Caching / rendering strategy	N/A	Partial	N/A	Partial	● Polished

Every customer and admin route. "Data" = connected to a real Supabase table/RPC, not a stub. RTL column flags any confirmed direction bug on that route specifically.

Customer routes (`app/[locale]**`)

ROUTE	PURPOSE	EXISTS	FUNCTIONAL	DATA	EN
<code>/</code>	Homepage	Y	Y	Y	Y
<code>/c</code>	Category index	Y	Y	Y	Y
<code>/c/[categorySlug]</code>	Category listing	Y	Y	Y	Y
<code>/box</code>	Green Box listing	Y	Y	Y	Y
<code>/p/[productSlug]</code>	Product detail	Y	Y	Y	Y
<code>/search</code>	Text search	Y	Y	Y	Y
<code>/api/search/suggest</code>	Suggestion API	Y	Y	Y	n/a
<code>/cart</code>	Cart	Y	Y	Y	Y
<code>/checkout</code>	Checkout	Y	Y	Y	Y
<code>/account/orders/[n]/confirmation</code>	Order confirmation	Y	Y	Y	Y
<code>/account/orders</code>	Order list	Y	Y	Y	Y
<code>/account/orders/[n]</code>	Order detail + tracking + reorder	Y	Y	Y	Y
<code>/account</code>	Redirect to profile	Y	Y	n/a	n/a
<code>/account/profile</code>	Profile edit	Y	partial	Y	Y
<code>/account/addresses</code>	Address list	Y	Y	Y	Y

ROUTE	PURPOSE	EXISTS	FUNCTIONAL	DATA	EN
/account/addresses/new	Add address	Y	Y	Y	Y
/account/addresses/[id]/edit	Edit address	Y	Y	Y	Y
/account/loyalty	Loyalty balance + history	Y	Y	Y	Y
/account/subscriptions	Subscription list	Y	Y	Y	copy bug
/account/subscriptions/new	Create subscription	Y	Y	Y	copy bug
/account/subscriptions/[id]	Subscription detail	N	—	—	—
/auth/login	Login	Y	broken redirect	Y	Y
/auth/register	Register	Y	Y	Y	Y
/auth/forgot-password	Forgot password	N	—	—	—
global error.tsx / not-found.tsx / loading.tsx	Fallback UI	Y	Y	n/a	loading hardcoded EN

Admin routes (app/admin/**)

ROUTE	PURPOSE	EXISTS	FUNCTIONAL	DATA	CRUD
/admin/login	Admin login	Y	Y	Y	n/a
/admin	Dashboard	Y	Y	Y	n/a
/admin/staff	Staff / roles	Y	Y	Y	promote/demote only
/admin/products	Product list	Y	Y	Y	n/a
/admin/products/new	Create product	Y	Y	Y	create only

ROUTE	PURPOSE	EXISTS	FUNCTIONAL	DATA	CRUD	
/admin/products/[id]/edit	Edit product + images + box contents	Y	Y	Y	Y	z
/admin/categories (+ new/edit)	Categories CRUD	Y	Y	Y	no reactivate	z
/admin/boxes	Green Box list	Y	Y	Y	read + delegate	r
/admin/orders (+ [id])	Orders + status + payment verification	Y	verification silent-fails	Y	n/a	F
/admin/customers (+ [id])	Customers	Y	Y	Y	read-only (by design)	r
/admin/delivery-zones (+ new/edit)	Zones + coverage areas	Y	Y	Y	areas: no edit	z
/admin/delivery-slots (+ new/edit)	Time slots + capacity	Y	Y	Y	Y	z C r
/admin/payments	Payment methods	Y	Y	Y	update only	r
/admin/loyalty	Loyalty settings + accounts list	Y	Y	Y	settings: update only	r
/admin/loyalty/[profileId]	Per-customer adjustment	Y	Y	Y	append-only ledger (correct)	s F
/admin/subscriptions	Subscription list + filter	Y	Y	Y	n/a	r
/admin/subscriptions/[id]	Detail + status + manual renewal	Y	Y	Y	n/a	F
/admin/content	Banners + store settings	Y	store-name is dead	Y	banners: no edit	z (r /

§25 Component Inventory

Major reusable components. Storefront and admin each have their own component library; `components/ui/*` is shared.

Shared primitives — `components/ui/*`

COMPONENT	USED WHERE	FUNCTIONAL	REUSABLE	UI QUALITY / NEEDS CHANGES
<code>app-image.tsx</code>	Every image site-wide	Y	Y	Correctly special-cases the placeholder host to bypass a known optimizer flake. Solid.
<code>badge.tsx</code>	Products, orders, addresses, admin tables	Y	Y (8 tones)	Consistent; <code>gold</code> tone uses a raw hex instead of a token — minor.
<code>button.tsx</code>	Everywhere	Y	Y	High quality: <code>forwardRef</code> , built-in spinner, <code>aria-busy</code> .
<code>card.tsx</code>	Nearly every page	Y	Y (3 tones)	Storefront always uses <code>glass</code> ; admin almost always uses <code>flat</code> — the tone split <i>is</i> the visual-inconsistency root cause (see §5).
<code>empty-state.tsx</code> / <code>ErrorState</code>	Every list route	Y	Y	Good reuse; <code>ErrorState</code> itself is under-used (1 call site).
<code>form-message.tsx</code>	All forms	Y	Y	Renders whatever text it's given — several callers pass raw error codes instead of real messages (caller bug, not component bug).
<code>input.tsx</code> / <code>label.tsx</code>	All forms	Y	Y	Correct <code>htmlFor</code> / <code>id</code> pairing verified.
<code>select.tsx</code>	Checkout, addresses, subscriptions	Y	Y	Nice detail: RTL-aware dropdown-arrow background position.
<code>pagination.tsx</code>	Category listing only	Y	Y	Correct RTL chevron swap. Not used by <code>/search</code> — gap.
<code>price-display.tsx</code>	Product cards, gallery, orders	Y	Y	Forward-compatible <code>compareAt</code> prop, no DB column yet (deliberate).
<code>skeleton.tsx</code>	Nowhere	N	—	Fully built (<code>ProductCardSkeleton</code> , <code>ProductGridSkeleton</code> , <code>TableRowSkeleton</code>) but never imported anywhere — dead code.

Storefront — `components/storefront/*`

COMPONENT	USED WHERE	FUNCTIONAL	UI QUALITY / NEEDS CHANGES
<code>checkout-form.tsx</code>	Checkout	Y	Broken discards <code>create_order</code> 's specific error messages.
<code>login-form.tsx</code>	Login	N	Broken post-login redirect chain (\$4 friction #2).
<code>product-card.tsx</code>	Home, category, box, search, related — heavily reused	Y	Hardcodes a "vegetables" fallback image regardless of the product's real category.
<code>reorder-button.tsx</code>	Order detail	Y	Hardcoded <code><</code> character breaks in one locale; conflates real errors with "nothing available."
<code>subscription-create-form.tsx</code>	New subscription	Y	Hardcoded weekday-name arrays and one bilingual inline string, bypassing <code>messages/*.json</code> entirely.
<code>profile-form.tsx</code>	Profile	partial	No error-state rendering at all.
<code>address-form.tsx</code>	Address new/edit	Y	Shows a raw error code string to the user.
<code>locale-switcher.tsx</code>	Header, mobile nav	Y	Drops the query string when switching languages.
<code>search-box.tsx</code>	Header (×2), mobile nav, search page	Y	Real debounced API call, click-outside handling — solid.
<code>cart-item-row.tsx</code>	Cart	Y	No per-item notes UI despite the backing service existing; quantity buttons have non-descriptive <code>aria-label="+"/"-"</code> .
<code>order-tracking-timeline.tsx</code>	Order detail	Y	Good: separate desktop (horizontal) / mobile (vertical) layouts.
<code>site-header.tsx</code> / <code>site-footer.tsx</code> / <code>mobile-nav.tsx</code>	Root layout	Y	Footer hardcodes "Vodafone Cash"/"InstaPay" badges disconnected from the real <code>payment_methods</code> table.
All other storefront components (register/logout/logo/address-list/subscription-card/product-gallery/product-purchase-form/auth-shell/cart-link)	Various	Y	No material issues found.

COMPONENT	USED WHERE	FUNCTIONAL	UI QUALITY / NEEDS CHANGES
<code>admin-nav.tsx</code>	Dashboard layout	Y	Mobile hamburger icon (<code>layout.tsx:21</code>) has no <code>onClick</code> /state — dead affordance implying a toggle that doesn't exist.
<code>order-status-control.tsx</code>	Order detail	Y	Best-practice example in the codebase: real try/catch + error display.
<code>payment-verification-control.tsx</code>	Order detail	silent-fails	Broken : no try/catch, unlike its sibling above.
<code>box-contents-editor.tsx</code>	Box product edit	Y	Save is delete-then-insert, non-atomic — can wipe contents on partial failure.
<code>product-image-manager.tsx</code>	Product edit	Y	Delete has no confirm dialog, unlike <code>archive-product-button.tsx</code> / <code>deactivate-button.tsx</code> .
<code>store-settings-form.tsx</code>	Content page	dead field	"Store name" writes real data that's never read anywhere on the storefront; error path shows nothing.
<code>banner-manager.tsx</code>	Content page	Y	Create/Delete/Toggle only — no edit-in-place.
<code>category-form.tsx</code>	Categories new/edit	Y	Image field is a raw URL text input, unlike the full uploader products get.
<code>deactivate-button.tsx</code>	Categories, zones, slots	Y	Genuinely reusable generic abstraction across 3 resources — a good pattern to extend.
<code>staff-manager.tsx</code>	Staff page	Y	Self-demote button hidden client-side only (harmless — RPC still enforces the real guard).
<code>subscription-controls.tsx</code>	Subscription detail	Y	Best example of RPC-result UX in the codebase — try/catch plus outcome display.
All other admin form components (<code>slot-form</code> , <code>zone-form</code> , <code>payment-method-form</code> , <code>area-manager</code> , <code>loyalty-settings-form</code> , <code>loyalty-adjustment-form</code> , <code>product-form</code> , <code>archive-product-button</code> , <code>admin-login-form</code> , <code>admin-logout-button</code> , <code>stat-card</code>)	Various	Y	No material issues found.

All 23 tables in the live `public` schema. Every table has RLS enabled with at least one policy — confirmed live via `pg_policies`, not read from migration files. Row counts as of the audit date; nearly all business-table content is test data (see §27).

TABLE	PURPOSE	RLS	CUSTOMER ACCESS	ADMIN ACCESS	KEY F
<code>profiles</code>	User identity / role	On	own row (select/update)	ALL	hand admi is_a
<code>categories</code>	Catalog taxonomy	On	select (active only)	ALL	—
<code>products</code>	Catalog items	On	select (available + active category)	ALL	sear sear
<code>product_images</code>	Product media	On	select (public)	ALL	—
<code>box_items</code>	Green Box bundle composition	On	select (public)	ALL	enfo
<code>delivery_zones</code>	Delivery pricing/zones	On	select (active)	ALL	—
<code>delivery_areas</code>	Governorate/city/area → zone	On	select (active + zone active)	ALL	—
<code>delivery_time_slots</code>	Delivery windows + capacity	On	select (active)	ALL	crea _gen
<code>addresses</code>	Customer addresses	On	ALL (own)	select	enfo
<code>payment_methods</code>	Payment options	On	select (active)	ALL	crea
<code>subscriptions</code>	Recurring order config	On	ALL (own)	select	admi gene
<code>subscription_items</code>	Subscription line items	On	ALL (via subscription join)	select	—
<code>carts</code> / <code>cart_items</code>	Active cart	On	ALL (own)	<i>none by design</i>	crea

TABLE	PURPOSE	RLS	CUSTOMER ACCESS	ADMIN ACCESS	KEY F
orders	Orders	On	select (own)	select	crea upda
order_items	Order line items	On	select (own)	select	crea
order_status_history	Status audit trail	On	select (own)	select	upda
payments	Payment attempts	On	select (own)	select	crea reco
loyalty_settings	Loyalty program config (singleton)	On	select (public)	ALL	crea upda
loyalty_accounts	Points balance	On	select (own)	select	admi
loyalty_transactions	Points ledger	On	select (own)	select	same
settings	Generic key/value store	On	select (public)	ALL	—
banners	Homepage banners	On	select (active)	ALL	—

Storage: a single `media` bucket (public read), with admin-only insert/update/delete policies keyed to `is_admin()` — confirmed live. No Edge Functions are deployed; all business logic lives in Postgres functions + Next.js server actions.

§11 Database / RLS Security Audit

Every policy expression was read directly (not assumed) via live `pg_policies` queries, and cross-customer access was tested against the actual `using` / `with check` clauses. **No CRITICAL or HIGH severity issue was found.** Findings below are what remains.

MEDIUM • MIGRATION TRACKING DRIFT

`supabase_migrations.schema_migrations` stops at `0020`; migration `0021_schedule_subscription_renewal_cron.sql` is not recorded, yet its effect (the live `generate-subscription-orders` cron job) unquestionably exists in `cron.job`. A future `supabase db diff /reset` workflow would not know this migration was already applied and could silently skip or double-apply it.

MEDIUM • LEAKED PASSWORD PROTECTION DISABLED

Supabase Auth advisor `auth_leaked_password_protection` (WARN): customers can currently register with a password already present in a known-breach corpus (HaveIBeenPwned). One toggle in Auth settings.

MEDIUM • STALE LOYALTY DATA FROM AN ALREADY-FIXED BUG

Orders `3957e003` (subtotal 1050) and `01987b6a / 3bfb0333` (subtotal 1250 each) are `DELIVERED` with `loyalty_points_earned=0` despite qualifying for 100 points each — a pre-`0017` composite-NULL bug, now fixed for all orders going forward (verified live: an order delivered *after* the fix correctly earned 100 points). These three rows simply won't self-correct; harmless since all data here is test data, but would need a manual backfill if this were real customer history.

LOW • ADVISOR FLAG ON A PLATFORM-MANAGED FUNCTION

`rls_auto_enable()` shows a `PUBLIC / anon / authenticated` EXECUTE grant per the security advisor. Investigated directly: it backs Supabase's own platform event trigger `ensure_rls` (not authored by this project) and its return type is `event_trigger`, so Postgres refuses direct invocation regardless of the grant — not exploitable via the REST API. Benign; flagged only so a future audit doesn't mistake it for a project-introduced backdoor.

LOW • PROOF-OF-PAYMENT STORAGE PATH DOESN'T EXIST YET

`payments.proof_image_url` and `payment_methods.requires_proof` exist in schema, but both live payment methods have `requires_proof=false`, and `uploadMediaFile()` only accepts `products / categories / banners` folders — there is no code path for a customer to upload a payment-proof image, and the only storage INSERT policy is admin-only. Not a live vulnerability since the feature doesn't exist; flagged so its eventual storage-policy design gets explicit attention (a customer needs a scoped INSERT path, which the current admin-only bucket model doesn't provide).

LOW • PERFORMANCE ADVISORIES (NOT SECURITY)

110 instances of multiple permissive RLS policies stacking on the same table/role/action (e.g. `products_admin_all` + `products_public_select` both evaluate on every SELECT) — functionally correct (OR'd), pure per-row overhead at scale. Plus 18 unused indexes, expected at current test-data row counts.

What was specifically verified clean

- No table has RLS disabled, and no table has zero policies.
- `authenticated` has **SELECT-only** access (confirmed via `information_schema.role_table_grants`) on `orders`, `order_items`, `payments`, `loyalty_accounts`, `loyalty_transactions` — a customer cannot forge status, price, or balance directly; every such write exists only inside a SECURITY DEFINER function.
- Every owner-scoped policy (`addresses`, `orders`, `payments`, `loyalty_*`, `subscriptions`) resolves through `profile_id = auth.vid()` or an equivalent join — no cross-customer read/write path found anywhere.
- Every admin-only RPC (`admin_generate_subscription_order`, `admin_set_subscription_status`, `admin_adjust_loyalty_points`, `admin_set_profile_role`, `update_order_status`, `record_payment_verification`) re-checks `is_admin()` internally.
- Cron-only functions (`generate_subscription_orders`, `_generate_order_for_subscription`) have no `anon` / `authenticated` grant at all — not reachable via the API, only via the `postgres`-owned cron job.
- `is_admin()`, `normalize_arabic()`, `search_products()`, `search_suggestions()` intentionally carry public grants — all four are pure, side-effect-free reads needed by public-facing RLS/search, and each was inspected to confirm no data leakage.

Full per-route detail is in the Route Inventory. Below are the friction points found while tracing the actual customer path end-to-end — landing → browse → search → product → cart → checkout → order → account — each one a genuine dead end or misleading state, not a stylistic nitpick.

`checkout-form.tsx:195` · `checkout/actions.ts:51`

Checkout swallows every specific server error. The `create_order` RPC raises 17 distinct, actionable messages (e.g. "This delivery slot is fully booked for the selected date," "Order subtotal is below the minimum order amount for this zone"). The action captures the real message into `state.message`, but the form only checks `state.status === "error"` and always renders one generic "order failed" string — the specific reason is computed, transmitted, and then discarded. A customer whose slot is full has no way to understand why, or what to do differently.

`lib/auth/session.ts:34-41` · `auth/actions.ts:42` · `login-form.tsx`

Login does not honor `?next=` — the single highest-impact break found. Guests get redirected to `/auth/login?next=<path>` from both `requireAuth()` and `addToCartAction`, but the login page never reads that param, the form never forwards it, and the post-login redirect is hardcoded to the homepage (with a comment stating "Phase 1 has no protected /account area yet" — no longer true). A guest who taps "Add to cart," logs in, lands on the homepage instead of back at the product or their cart.

`checkout/page.tsx:27-33` · `account/subscriptions/new/page.tsx:23-56`

Three mislabeled CTA copy-paste bugs. The checkout empty-cart state's "go home" button is labeled with the `addAddress` translation key ("Add a new address"). Both empty states on the subscription-creation page reuse the `create` key ("Create a new subscription") as their button text, regardless of what the button actually does.

`box/page.tsx` · live DB: 0 of 68 products are `product_type='box'`

Green Box is a fully-built dead end today. The homepage's "Discover Green Box" CTA and the `/box` route both work exactly as coded — there is simply no box product to show, so every real visitor sees the empty state. This is a content gap, not a code gap.

`address-form.tsx:83-87` · `profile-form.tsx`

Failed saves are poorly or never surfaced. A failed address save literally renders "Save address: GENERIC" or "Save address: VALIDATION" — the raw internal error code, not a sentence. A failed profile save renders nothing at all; the component has no error branch.

`reorder-button.tsx:21-26`

Reorder conflates real errors with "nothing available." Any non-success result — including a genuine thrown exception — is shown as "None of these items are currently available," which actively misleads the customer about what actually happened.

`search/page.tsx:17` · `lib/services/catalog.ts:92,98`

Search silently caps at 20 results with no pagination or count. A query matching 50 products shows exactly 20 with no indication more exist.

locale-switcher.tsx:16

Switching AR→EN drops the current query string. A customer on page 2 of search results, or filtering a category, lands back at page 1 / no filter after switching language.

checkout-form.tsx:73,192

Checkout's headline "Total" is not the real total. It's computed as `subtotal - redemption` and shown prominently; delivery fee (unknown client-side until `create_order` actually runs) is disclosed only as a small footnote below it. The number the customer fixates on isn't what they'll be charged.

lib/services/subscriptions.ts:34 (getMySubscription)

Subscription detail view doesn't exist. The service function is fully built and has zero callers — there is no `/account/subscriptions/[id]` route, so customers only ever see the flat list card, never a dedicated detail page.

Assessed strictly from source (CSS custom properties, Tailwind classes, component code) — no rendering was performed. The brief's target: premium, modern, fresh, natural, sophisticated, 3D liquid-glass/glassmorphism, high-end e-commerce feel, primary `#84B63D`, dark `#1C3728`.

What already matches the direction

- **Brand colors are wired correctly, not ad hoc.** `--brand-500: #84b63d` and `--deep-700: #1c3728` match the brief exactly and flow through Tailwind v4's `@theme inline` block as first-class utilities (`app/globals.css:30,43,80-105`) — this is a real token system, not hardcoded hex scattered around.
- **The glass mechanism is genuinely well-built, not superficial.** `.glass` / `.glass-panel` / `.glass-hover` / `.glass-dark` (`app/globals.css:191-258`) combine real `backdrop-filter: blur(18-20px)` `saturate(...)`, a semi-transparent gradient fill, a visible 1px border, a layered gradient-border mask for a light-catching edge, and dual box-shadows for depth — this precisely matches "visible glass borders and subtle 3D depth."
- The storefront is **fully committed** to this treatment: every `Card` usage found in `app/[locale]/**` and `components/storefront/*` uses the glass tone (grepping for `tone="flat"` in storefront code returns zero matches).
- Global focus-visible ring, reduced-motion handling, and font loading are all handled once, centrally, and applied everywhere — a real cross-cutting design-system win.

What does not match — and where

- **The admin dashboard is a different, plainer design language.** Every primary admin data table uses `Card tone="flat"` (a plain bordered box, no glass) — dashboard, customers, categories, orders, delivery slots, subscriptions, products, loyalty list. Only login, the content page, and one card on the loyalty page opt into glass. `app/admin/(dashboard)/page.tsx` is the clearest single-file proof: its stat tiles render as glass while the "Recent Orders" table directly beneath them on the same screen is flat — two design languages stacked on one page.
- **Competing border-radius values with no shared token.** The `--radius-card: 1.25rem` token is used correctly in several places, but hero image containers hardcode a different, larger `rounded-[2rem]` instead (`app/[locale]/page.tsx:73,82`, `box/page.tsx:43,51`).
- **Shadow tokens exist but are frequently bypassed.** At least 19 files hand-tune one-off `rgba(...)` box-shadow values instead of reusing `--shadow-soft` / `--shadow-lifted` (e.g. `components/ui/button.tsx:9,11`). Color-consistent with the brand, but not systematized.
- `components/ui/badge.tsx:9` — the gold tone uses a literal `text-[#8a6a1f]` hex instead of the `--gold-400/500` tokens already defined two lines away in the theme.

Redesign priority

Needs redesign: every admin list/table page and most admin form-component shells (§8 lists them) — to either adopt the glass treatment or have a deliberately-scoped "admin uses a plainer utilitarian surface" decision made explicitly, rather than the current unplanned mix. **Can be reused as-is:** the entire glass CSS mechanism, the token system, `Card` / `Badge` / `Button` primitives, and every storefront page — none of this needs rebuilding, only wider/consistent application.

No live device testing was performed (no browser tool available) — findings are inferred from Tailwind breakpoint usage in source.

- **No confirmed broken layouts found** in either storefront or admin: every data table checked (customers, orders, categories, delivery slots, subscriptions) is wrapped in `overflow-x-auto` rather than being left to overflow the page; grids consistently step down column counts at `sm:` / `md:` / `lg:` breakpoints (e.g. product grids `grid-cols-2 sm:grid-cols-3 md:grid-cols-4` patterns); no fixed pixel widths without a mobile override were found.
- **Admin mobile nav hamburger is non-functional.** `components/admin/admin-nav.tsx` + `app/admin/(dashboard)/layout.tsx:21`: the hamburger icon has no `onClick` /state, and the nav below it has no `hidden` class — it already always renders as a horizontally-scrollable strip on mobile. The icon implies a toggle that doesn't exist; low-severity but confusing.
- Touch targets: several icon-only controls sit at 32–36px (`h-8 w-8` / `h-9 w-9`) — above WCAG's 24×24 minimum but below the commonly recommended ~44px comfortable target, on frequent-interaction surfaces like cart quantity/remove buttons.
- Order tracking timeline (`order-tracking-timeline.tsx`) deliberately implements separate horizontal (desktop) and vertical (mobile) layouts — a genuinely good responsive pattern, called out as a positive example.
- Glass effects (`backdrop-filter: blur`) are declared without a `@supports` fallback; on the very small slice of older mobile browsers without backdrop-filter support, glass surfaces would render as their plain background color rather than break — a graceful-enough degradation, not a functional bug.

§7 Bilingual / RTL Audit

The codebase's default pattern — Tailwind logical properties (`ms-` / `me-` / `ps-` / `pe-` / `start-` / `end-`) instead of physical (`ml-` / `mr-` / `left-` / `right-`) — is followed consistently: no stray physical-direction class was found anywhere in storefront code. Two genuine direction bugs exist against that otherwise-clean baseline:

- `app/[locale]/p/[productSlug]/page.tsx:38` — the breadcrumb chevron logic is *inverted* relative to every other directional icon in the codebase: `locale === "ar" ? ChevronRight : ChevronLeft`, while `mobile-nav.tsx:32` and `account/orders/page.tsx:16` both correctly use the opposite mapping. The product-page breadcrumb arrows point the wrong way in both locales relative to the site's own established convention.
- `components/storefront/reorder-button.tsx:48` — a literal `←` character is hardcoded regardless of locale, unlike every other directional element in the app which swaps via a locale check.

i18n completeness

All `orders.status.*`, `orders.paymentStatusLabels.*`, and `subscriptions.status.*` keys used by `lib/ui/status.ts` and order/subscription pages were cross-checked against the actual `OrderStatus` / `PaymentStatus` / `SubscriptionStatus` string-union types in `types/database.ts` — no missing status label found. `messages/en.json` and `messages/ar.json` are structurally identical (same key set).

Dead translation keys (defined in both files, never referenced): `checkout.selectRequired`, `cart.notesPlaceholder`, `product.noImage`, `common.language`, `common.loading`.

Hardcoded strings that bypass i18n entirely: `app/[locale]/c/[categorySlug]/page.tsx:57` (literal `"منتج"` / `"products"` ternary), `subscription-create-form.tsx:19-20,84` (hardcoded weekday-name arrays and one bilingual inline string), `app/[locale]/loading.tsx:18` (hardcoded English `"Loading"` shown even to Arabic users, despite an unused `common.loading` key existing for exactly this).

§8 Admin Frontend Audit

Full per-route detail is in the Route Inventory and Component Inventory. Every admin mutation traced — product/category/image/zone/slot/payment-method/banner/settings CRUD via RLS `*_admin_all` policies, and every RPC-backed action (order status, payment verification, loyalty adjustment, subscription status/renewal, role change) via an internal `is_admin()` check — resolves to a real, confirmed-live server-side authorization boundary. No admin mutation was found relying on client-side trust alone.

The two real functional defects

- **Payment verification fails silently.** `components/admin/payment-verification-control.tsx:35-37` calls the verification action with no try/catch, unlike the near-identical `order-status-control.tsx:38-44`. If `record_payment_verification` throws (invalid state, race condition), the admin sees nothing — no error, no indication the click did anything.
- **Box contents save is not atomic.** `adminSetBoxContents()` deletes all existing `box_items` then re-inserts — if the insert fails partway (e.g. a bad product id), the box is left with zero contents and no rollback.

CMS content-editable surface (full audit, see also §16)

Exactly two things are admin-editable today: banners (create/delete/toggle, no edit-in-place) and one settings row, `store_info` (`store_name`, `contact_phone`, `contact_email`, `whatsapp_phone`). **The `store_name` field is a confirmed dead control** — it persists to the database but no storefront component reads it; the site name shown in the header/footer/copyright comes from a next-intl translation key instead. An admin editing "اسم المتجر" sees zero visible effect anywhere.

Customer-visible copy that reads as CMS content but is actually hardcoded in `messages/*.json` (developer-only edits): the homepage hero eyebrow/title/subtitle/CTAs, the 4-card trust strip, the "why Green Box" section, Green Box/loyalty/subscription teaser copy, the final CTA banner, product-page trust badges, and the footer's "Vodafone Cash"/"InstaPay" badges — the latter are hardcoded literal strings not even backed by a translation key, and don't reflect the real admin-managed `payment_methods` list at all.

§9 Backend Architecture Audit

Every money/loyalty/role/subscription-status mutation goes through a SECURITY DEFINER Postgres function that performs its own internal authorization check — this is a deliberate, consistently-applied architecture, confirmed function-by-function (see §11 for the full table). Plain-CRUD tables (products, categories, banners, zones, slots, payment methods, box_items, product_images) instead rely on RLS `*_admin_all` / `*_public_select` pairs. No Edge Functions are deployed; there is no service-role client used anywhere in request-handling code (confirmed: `SUPABASE_SERVICE_ROLE_KEY` is reserved for one-off infra scripts only, per its own env-file comment). All server actions and services follow one convention: `page.tsx` (Server Component, fetches via `lib/services/*.ts`) → `actions.ts` ("use server") → service function → Supabase table or RPC. No deviations from this pattern were found except `app/admin/(dashboard)/boxes/page.tsx`, which queries `products` directly instead of through `lib/services/catalog.ts` — functionally harmless (same RLS applies) but breaks the stated convention.

Customer

- **Registration / Login / Logout:** real Supabase Auth (`signUp` / `signInWithPassword` / `signOut`), both fully wired — **Complete** .
- **Password reset:** **Missing** entirely — no route, no email flow, no link to it from the login page.
- **Session handling:** cookie-based via `@supabase/ssr` , correct.
- **Protected routes:** `requireAuth()` gates `/account/**` , `/checkout` ; correctly redirects guests — but the redirect destination after login is broken (see §4, friction #2).

Admin

- **Admin login:** real, with a role check after sign-in — **Complete** .
- **Admin authorization:** `requireAdmin()` gates page rendering in `app/admin/(dashboard)/layout.tsx` (defense-in-depth); the real boundary is `is_admin()` inside RLS/RPCs, confirmed on every single admin mutation path traced.
- **Roles:** exactly two (`customer` / `admin`) — deliberately kept simple, matches the actual need.
- **Staff management:** promote-by-email / demote, fully functional and RPC-authorized.
- **Last-admin protection:** live-verified — attempting to demote the sole admin is rejected by the database with `Cannot remove the last administrator` .
- **Self-escalation protection:** a customer cannot promote themselves — enforced independently by (a) `role` column revoked from `authenticated` at the grant level, (b) the `profiles_prevent_role_self_change` trigger, and (c) `admin_set_profile_role` 's own `is_admin()` check. Three redundant layers.
- **Permission boundaries:** no gap found — every admin-only table and RPC resolves to an `is_admin()` check, live-confirmed.

Unauthorized-access test result: a non-admin authenticated user hitting any admin server action directly (bypassing the UI) would still be rejected at the RLS/RPC layer regardless — the app-layer `requireAdmin()` guard is a UX convenience, not the actual security boundary, and the actual boundary holds independently everywhere it was checked.

- **Products:** 68 real rows (1 test row), full admin CRUD, correct RLS. **Complete** .
- **Categories:** 7 rows, full admin CRUD. **Complete** .
- **Product images:** mechanism fully built (upload/delete/set-primary, reflected correctly on the storefront) — but `product_images` has zero rows, so every product currently shows a stock placeholder. **Polish (needs content)** .
- **Primary image:** schema and admin control both work; storefront correctly prefers it via `find(is_primary) ?? images[0]` .
- **Availability:** single `is_available` boolean, consistently enforced in cart/checkout/subscription-renewal availability checks.
- **Pricing:** always server-computed from live `products.price` at cart-summary/checkout/reorder time — never client-supplied, never stale.
- **Discounts:** no discount/compare-at-price mechanism exists yet; `PriceDisplay` has a forward-compatible `compareAt` prop with no backing column — deliberate placeholder, not a bug.
- **Product metadata (SEO fields):** `meta_title_*` / `meta_description_*` exist and are used on category/product pages.
- **Box contents / Green Box products:** mechanism fully functional (create/contents/quantities/ordering all wired) — but zero real box products exist, so the feature is invisible to customers today. **Blocked by content** .
- **Reservation requirements:** `requires_reservation` flag exists, admin can set it, a storefront badge surfaces it — but no cutoff/lead-time/confirmation workflow exists, and none was invented (correctly, per the standing business-decision gap).
- **Related products:** real, same-category query. **Complete** .
- **Search:** real RPC-backed full-text search with Arabic normalization; capped at 20 results with no pagination.
- **Filtering / sorting:** not found anywhere in the category or search UI — no price/availability filter, no sort-by-price/newest control exists on any listing page.

Traced live: Product → Cart → Cart Items → Checkout → Order → Payment → Status → Delivery → Tracking → Reorder.

- **Validation:** address ownership, delivery-area/zone active state, delivery date (no past dates), slot capacity, minimum order amount, and loyalty-redemption bounds are *all* enforced inside `create_order()` itself — not just in the UI. Verified via the live function body.
- **Minimum order amount:** enforced server-side against the zone's configured minimum; re-checked against the *reduced* subtotal during subscription renewal if a product became unavailable.
- **Availability:** checked at order-creation and at subscription-renewal time; renewal silently drops unavailable items rather than failing the whole cycle.
- **Delivery zones / slots:** real, admin-configurable, capacity-limited with a row-level lock (`for update`) preventing double-booking under concurrent checkouts — confirmed in the live function source.
- **Payment:** manual/proof-based only (2 seeded methods); admin verification flips `payment_status` correctly, live-confirmed via a real verified payment row.
- **Order status progression:** fixed sequence enforced server-side, terminal states (`DELIVERED` / `CANCELLED`) block further changes, full audit trail in `order_status_history` with real notes.
- **Cancellation:** supported via the status RPC; correctly reverses redeemed loyalty points (live-verified ledger entry: `REVERSED +100`).
- **Reorder:** correctly uses current prices (cart never stores a price, always looks it up live) and correctly skips unavailable/deleted products — but see §3–4 for its error-messaging bug.
- **Loyalty interaction:** earn-on-delivery and redeem-at-checkout both verified live and reconcile exactly against the ledger.
- **Subscription interaction:** renewal-generated orders correctly derive the delivery zone from the subscription's live address (not a possibly-stale cached zone id), and never auto-redeem loyalty points — a deliberate choice to avoid inventing an unconfirmed payment-automation rule.

Frontend gap, not a backend one: checkout's UI discards the specific validation failure reason the backend already computed and sends back (see §3–4). The business logic itself has no missing rule found.

Every mechanism below was exercised against the live database this audit, not inferred from code alone.

- **Earning:** confirmed live — a real order delivered after the governing bugfix earned exactly `floor(subtotal/1000)*100` points.
- **Redemption:** confirmed live at checkout; over-redemption beyond the current balance is blocked server-side.
- **Balance:** single `loyalty_accounts.points_balance`, never directly writable by anyone — only through the RPC layer.
- **Transactions (ledger):** every earn/redeem/reverse/adjust event produces a row; summing all transactions for the audited test account reproduces its current balance exactly.
- **Admin adjustments:** live-tested end-to-end — `+50` then `-50` both applied correctly, each requiring a non-empty reason, each producing a `type='ADJUSTED'` ledger row.
- **Negative balance protection:** live-tested — an adjustment or redemption that would drop the balance below zero is rejected by the database with an explicit error.
- **Cancellation reversal:** live-confirmed — cancelling an order with redeemed points restores them with a ledger row explaining why.
- **Checkout integration:** real, server-validated, capped at the configured minimum-redeemable-points floor.
- **Customer UI:** balance + history view, **Complete**.
- **Admin UI:** settings + per-customer adjustment with full history, **Complete**.

Implemented

- Creation, weekly cadence (day-of-week), delivery date computation on create.
- Renewal: a real, live, scheduled `pg_cron` job (daily 06:00 UTC) invoking a SECURITY DEFINER function locked to `postgres` / `service_role` only — not reachable via the public API.
- Order generation: re-validates status/address/zone/slot-capacity/product-availability/minimum-order at generation time, not just at creation time.
- Duplicate prevention: a real database-level unique partial index on `(subscription_id, delivery_date)` — not just an application-side check — makes a true duplicate impossible even under concurrent/overlapping cron runs.
- Status: pause/resume/cancel, both customer- and admin-initiated, correctly gated.
- Admin management: list/filter, detail view, manual "generate now" trigger with a real outcome/reason returned.
- Customer management: list, pause/resume/cancel.

Business decision still required

- **Payment/auto-charge behavior on renewal** — deliberately not implemented; renewal-generated orders never auto-redeem loyalty points and follow the same manual payment-status flow as any other order. This was a conscious choice to avoid inventing a payment-automation rule the business hasn't confirmed.
- **Subscription detail view for customers** is unbuilt (service function exists, no route) — not a business decision, just unfinished, listed here because it's the one piece of subscriptions left incomplete.

Full detail already given in §8. Summary: the admin-editable surface is exactly two things — banners (image/title/link/order/active, create-only) and one `store_info` settings row (name/phone/email/WhatsApp). No page/section builder, no publish/unpublish workflow beyond the banner active toggle, no per-language content split beyond the two title fields banners already have. The single most consequential finding: the "store name" admin field is **dead** — it saves, but nothing on the storefront reads it. A large amount of genuinely marketing-owned copy (hero, trust strip, teasers, final CTA, footer payment badges) lives in developer-only JSON translation files rather than anything an admin can touch.

§17 Images / Assets Audit

- **The single biggest asset finding is architectural, not cosmetic.** Live data confirms `product_images` = 0 rows across all 68 products, `categories.image_url` = null for all 7 categories, and `banners` = 0 rows. **100% of storefront imagery today is a fallback**, not real content.
- That fallback (`lib/media/placeholders.ts`) is backed by `loremflickr.com` — a random-keyword public stock-photo proxy, not curated imagery, with no commercial license guarantee. It's explicitly documented in code as a temporary measure, and is served **unoptimized** (bypasses Next's image pipeline entirely, per a code comment noting an observed optimizer reliability issue against this specific host).
- **Category-mismatched fallbacks:** of 17 category-specific placeholder keywords defined (chicken, dairy, bakery, fruits, grocery, etc.), only 4 are ever actually invoked. `product-card.tsx:25` hardcodes `isBox ? "greenBox" : "vegetables"` for *every non-box product regardless of its real category* — concretely, any of the 7 real chicken products without an uploaded photo (i.e. all of them right now) shows a stock **vegetable** photo. This is a content-accuracy defect, not just "no photo yet."
- 13 of the 17 defined placeholder keys are dead code, never called.
- Alt text is otherwise handled well: content-bearing images consistently carry a real localized `alt` ; decorative images correctly use `alt=""` . No missing alt text found.
- `public/` is completely empty — no local logo file, no OG image, nothing beyond the auto-generated `favicon.ico` . No duplicate or oversized local assets exist (nothing to duplicate).
- `components/ui/skeleton.tsx` is fully built but never used anywhere — dead code, not an asset per se, but relevant to the "temporary placeholder" theme.

Where real professional imagery is needed before launch: every one of the 68 real products (currently 0 have a photo), at least one category hero image per active category, and at least one real homepage/Green Box banner.

ASPECT	PRESENT	DETAILS / GAP
Sitemap	Y, but broken value	Dynamically generated from live categories/products for both locales — solid implementation — but <code>NEXT_PUBLIC_SITE_URL</code> is unset, so it falls back to the literal <code>https://greenbox.example</code> placeholder domain in every emitted URL.
Robots.txt	Y, incomplete	Correctly disallows <code>/admin</code> , <code>/api</code> , <code>/*/*account</code> , <code>/*/*checkout</code> , <code>/*/*cart</code> — but does not disallow <code>/*/*search</code> , a parameterized low-value route that's fully crawlable today.
Per-page metadata	Partial	Category and product pages get real title/description from CMS fields. Home, cart, checkout, search, and every <code>/*/*account/**</code> and every admin page have no per-page metadata at all.
Canonical URLs	Absent	Zero <code>alternates.canonical</code> anywhere in the codebase.
hreflang / locale alternates	Absent	Despite a fully bilingual AR/EN setup, no <code>alternates.languages</code> exists anywhere linking the two locale versions of a page for search engines.
Open Graph	Minimal	Only the product page sets <code>openGraph</code> , and only the <code>images</code> field — no <code>og:title</code> / <code>og:description</code> / <code>og:type</code> / <code>og:site_name</code> anywhere.
Twitter Card	Absent	No <code>twitter</code> metadata key found anywhere in the repo.
Structured data (JSON-LD)	Partial	A correct <code>Product</code> / <code>Offer</code> schema exists on the product page (price + availability). No <code>Organization</code> / <code>WebSite</code> schema on the home page, no <code>BreadcrumbList</code> despite a visible breadcrumb UI.
Admin noindex	Absent	Robots.txt disallow prevents crawling but not indexing of already-discovered URLs; no <code>robots: {index:false}</code> meta exists as defense-in-depth.
<code>metadataBase</code>	Absent	Not set in any layout.

§19 Performance Audit

- **No caching layer anywhere — every page is forced fully dynamic.** The Supabase server client calls `cookies()` on every request, which forces dynamic rendering for any Server Component that uses it — effectively every data-driven page. No `unstable_cache`, no `revalidate`, no route-level caching exists anywhere. Category/product listings change infrequently and are exactly the kind of data ISR/data-cache exists for, but every request re-hits Supabase from zero.
- **Duplicate auth round-trips per request.** `supabase.auth.getUser()` — a real network call, not a local JWT decode — runs twice on protected pages (once in the header, once in `requireAuth`); `requireAdmin()` similarly does its own independent `auth.getUser()` + a `profiles` query on every single admin page render.
- Several `fill`-mode images omit `sizes` (avatars, thumbnails), defaulting to a `100vw` assumption and over-fetching relative to their actual rendered size — while the main product-card/gallery/hero images correctly set it.
- Placeholder images are served fully unoptimized (bypass Next's image pipeline) — and today, that's effectively all storefront imagery (§17).
- **Positives:** no raw `<img>` tags anywhere (consistent `next/image` / `AppImage` use); no code-splitting need currently (components are modestly sized); Server/Client component boundaries are drawn correctly everywhere checked; dependency footprint is lean (no heavy client libraries); parallel data-fetching (`Promise.all`) is used correctly wherever a page needs multiple independent queries; font loading uses `next/font` consistently across all three root layouts.

§20 Accessibility Audit

- **Medium — mobile nav drawer lacks modal semantics.** No `role="dialog"` / `aria-modal`, no focus trap, no focus restore, no Escape handler; its three buttons (hamburger, backdrop, close) share one identical "Menu" `aria-label` even though two of them close, not open.
- **Medium — search suggestions aren't a real combobox.** No `role="combobox"` / `listbox` wiring, no `aria-expanded` / `aria-activedescendant`, no arrow-key navigation — keyboard-only users cannot navigate suggestions, only mouse.
- **Low — non-descriptive labels.** Cart quantity buttons use `aria-label="-"` / `"+"` (announced as "minus"/"plus," not "decrease/increase quantity").
- **Low/Medium — contrast risk on dark surfaces.** `text-white/40` tagline text on the dark auth-shell background is estimated around 3.5:1, likely below the 4.5:1 AA threshold for small text.
- **Low/Medium — touch targets.** Several icon-only controls at 32–36px, below the ~44px comfortable mobile target (still above the WCAG AA minimum of 24px).
- **Positives:** no `<div onClick>` anti-pattern found anywhere — every interactive element is a real button/link; label/input `htmlFor` / `id` pairing verified correct everywhere checked; a global `prefers-reduced-motion` rule collapses all animation site-wide; a global `:focus-visible` ring is defined once and applied consistently; native `<select>` / `<textarea>` are used rather than custom-built reimplementations, inheriting keyboard support for free; `eslint-config-next/core-web-vitals` (which bundles baseline `jsx-a11y` rules) is active and passes clean.

§21 Testing / Build Audit

CHECK	RESULT	DETAIL
<code>npx tsc --noEmit</code>	0 errors	Clean across the entire project.
<code>npx eslint .</code>	0 warnings	Clean across the entire project, including baseline accessibility rules.
<code>npx next build</code>	Success	All 56 routes compiled and registered correctly (32 customer, 24 admin/system).

Existing tests: none. A repo-wide search for `*.test.*`, `*.spec.*`, `__tests__/*`, and every common runner's config file (Jest, Vitest, Playwright, Cypress) returned zero matches. `package.json` installs no test framework and defines no `test` script. This is not "partially configured" or "broken" — testing is entirely absent as a discipline in this project today. No integration tests, no RLS tests, no end-to-end tests exist.

Explicitly stated per the audit brief: no browser automation tool was available in this audit, and none was used. Every UI/visual/responsive/accessibility finding above is derived from reading source code, never from an actual rendered page.

§22 Production Readiness Audit

CLASSIFICATION	ITEM
Ready	Core commerce loop (browse → cart → checkout → order → tracking), loyalty engine, subscription renewal engine, admin operations (orders, staff, zones/slots, loyalty adjustment), and the entire RLS/authorization model.
Needs fix before launch	Checkout/subscription error messages (discarded server errors), login <code>?next=</code> redirect chain, payment-verification silent-failure, <code>NEXT_PUBLIC_SITE_URL</code> unset, robots.txt missing <code>/search</code> disallow, forgot/reset password flow.
Can be deferred	Cart item notes, subscription detail view, box-content-save atomicity, banner edit-in-place, payment-method Create/Delete UI, category reactivate buttons, canonical/hreflang/Twitter metadata, caching/ISR, admin visual redesign.
Business decision required	Subscription payment/auto-charge behavior (currently, correctly, left unautomated); chicken/reservation cutoff & lead-time policy; stock/inventory model beyond boolean availability; refund/cancellation policy.
Blocked by environment / content	Real product/category/banner photography (0 uploaded today); at least one real Green Box bundle product; real WhatsApp number confirmation; a production domain to put in <code>NEXT_PUBLIC_SITE_URL</code> ; leaked-password protection toggle in Supabase Auth settings; test/QA data cleanup (every business-table row today traces to one of two <code>mailinator.com</code> test accounts).

Only incomplete or missing work. Nothing already marked complete above is repeated here.

A. Critical

- None. No CRITICAL-severity item was found anywhere in this audit — frontend, backend, or database.

B. High Priority

- Checkout and subscription-creation forms discard the specific, already-computed server error message and always show a generic failure — `checkout-form.tsx:195`, `subscription-create-form.tsx:109`.
- Login does not honor `?next=` — guests always land on the homepage after signing in instead of back where they were headed.
- Payment verification control has no error handling — failures are completely silent to the admin.
- `NEXT_PUBLIC_SITE_URL` is not configured — the sitemap emits URLs on the placeholder domain `https://greenbox.example`.
- No password reset flow exists at all.
- Zero test coverage of any kind (no framework installed, no test files).
- Zero real product/category/banner photography — every image today is a random third-party stock photo, several mismatched to the product's actual category.
- Green Box has no real bundle product — the feature is fully built but permanently empty.

C. Medium Priority

- Migration `0021` (cron schedule) is live but absent from the tracked migration history.
- Box-contents save is non-atomic and can wipe a box's contents on partial failure.
- Three mislabeled/copy-paste CTA bugs (checkout empty-cart, subscription-new empty states ×2).
- Raw error codes shown to users on address save failure; profile save has no error state at all.
- Product/category admin edit forms give no "saved" confirmation.
- Banners and delivery zones/areas cannot be edited in place (create+delete only, or no edit at all for areas).
- Stale `loyalty_points_earned=0` on 3 historical orders from an already-fixed bug (cosmetic on test data; would need a backfill decision if this were real history).
- Robots.txt does not disallow `/search`.
- No canonical URLs or hreflang alternates anywhere, despite a fully bilingual site.
- Admin dashboard's visual language (flat) doesn't match the storefront's (glass) — sometimes both appear on one screen.

D. Low Priority

- Search has no pagination past its 20-result cap.
- Locale switcher drops the current query string.
- Reorder button hardcodes a directional arrow character and conflates real errors with "nothing available."
- Product-detail breadcrumb chevron direction is inverted relative to the rest of the site.

- Cart item notes field exists in the backend with no UI ever calling it.
- Subscription detail view for customers is unbuilt (service function is dead code).
- Product image delete has no confirmation dialog (inconsistent with archive/deactivate elsewhere).
- Category image field is a raw URL input rather than an uploader.
- Payment methods support Update only, no Create/Delete UI.
- No "reactivate" button for archived products / deactivated categories, zones, or areas.
- Dead code: `components/ui/skeleton.tsx`, `getMySubscription()`, 5 unused i18n keys, 13 of 17 placeholder image keys.

E. Visual / UI Polish

- Admin dashboard needs either a deliberate glass-treatment pass or an explicit, documented decision to keep it plain.
- Two competing large-border-radius values in use (`--radius-card` token vs. hardcoded `rounded-[2rem]`).
- 19+ files hand-tune one-off box-shadow values instead of the existing shadow tokens.
- One hardcoded hex color (badge gold tone) bypassing the token system.
- Mobile hamburger icon in the admin nav is a non-functional decorative affordance.
- Touch targets on several icon-only buttons sit below the ~44px comfortable mobile size.
- Mobile nav drawer and search-suggestions dropdown both lack proper modal/combobox accessibility semantics.

F. Business Decisions

- Subscription payment/auto-charge behavior on renewal.
- Chicken/reservation cutoff and lead-time policy.
- Stock/inventory model — is boolean availability sufficient long-term, or is quantity tracking needed?
- Refund/cancellation policy beyond the current placeholder `REFUNDED` payment status.
- Real WhatsApp contact number (a discrepancy exists between two numbers seen in earlier source material; unresolved).

G. Deployment / Environment

- `NEXT_PUBLIC_SITE_URL` must be set to the real production domain.
- Leaked-password protection should be enabled in Supabase Auth settings.
- Migration-history drift on `0021` should be reconciled before any future `supabase db diff /reset` workflow.
- All test/QA data (every business-table row currently traces to one of two `mailinator.com` accounts) should be purged before go-live.
- No CI/CD pipeline, Dockerfile, or hosting config exists in the repo — deployment process is currently undefined.

01 Global Design System

Extend the glass treatment to admin, or explicitly scope glass to storefront-only and give admin its own deliberate flat/utilitarian system. Consolidate border-radius (`rounded-[2rem]` → a token) and shadow usage (hand-tuned rgba → `--shadow-soft` / `--shadow-lifted`). Move the badge gold-tone hex into the token scale.

02 Homepage

Upload at least one real banner so the hero stops defaulting to a stock photo. No structural work needed otherwise.

03 Navigation

Wire the admin mobile hamburger to an actual toggle, or remove it. Fix the locale switcher to preserve the current query string.

04 Categories

Add the missing `"منتج" / "products"` i18n key. No filters or sort controls exist — add if the catalog grows enough to need them.

05 Product Listing

Add pagination/result-count to `/search` (currently hard-capped at 20 with no indication more exist); reuse the existing `Pagination` component already used on category pages.

06 Product Details

Fix the inverted breadcrumb chevron direction. Fix the category-mismatched placeholder fallback (chicken products showing vegetable stock photos) once real photography or per-category fallback keys are wired in.

07 Search

Add keyboard navigation (arrow keys + `aria-activedescendant`) to the suggestions dropdown; wire proper `combobox` / `listbox` ARIA roles.

08 Cart

Either build UI for the existing (unused) per-item notes field, or remove the dead service function. Give quantity +/-buttons descriptive `aria-label` s.

09 Checkout

Highest-priority frontend fix in the whole audit: render `state.message` in the error branch of `checkout-form.tsx` so customers see the real, specific reason an order failed. Fix the mislabeled empty-cart CTA. Clarify that the displayed total excludes delivery fee until server confirmation.

10 Orders

No structural gaps found; polish only (see §3–4 for the reorder-button fixes).

11 Tracking

No gaps found — the desktop/mobile split implementation is solid as-is.

12 Account

Add an error-state branch to the profile form (currently renders nothing on failure). Replace raw error codes on the address forms with real messages.

13 Loyalty

No frontend gaps found.

14 Subscriptions

Fix the two mislabeled empty-state CTAs. Replace the hardcoded weekday arrays and inline bilingual string in `subscription-create-form.tsx` with real i18n keys. Build the missing customer-facing detail route, or delete the dead service function backing it.

15 Green Box

No code work required — this is a content task (create at least one real box product with real contents/pricing).

16 Authentication

Second-highest-priority frontend fix: wire `?next=` through login so users return to where they were headed. Build a forgot/reset-password flow via Supabase Auth's recovery API.

17 Admin

Add try/catch + error display to the payment-verification control. Make the box-contents save atomic. Add edit-in-place to banners. Add a real "saved" confirmation to product/category/zone edits. Remove or wire up the dead "store name" setting.

18 CMS

Decide which of the currently-hardcoded marketing sections (hero, trust strip, teasers, final CTA) are worth exposing as editable settings — don't build a full page builder, extend the existing generic `settings` pattern only where it earns its keep. Fix the footer's hardcoded payment-method badges to reflect the real `payment_methods` table.

19 Responsive

No broken layouts found; only the non-functional admin hamburger icon and a handful of under-44px touch targets need attention.

20 Arabic/RTL

Fix the two direction bugs (product breadcrumb chevron, reorder-button arrow). Localize the global loading spinner's text. Remove the 5 dead i18n keys or use them where they were clearly meant to be used (`common.loading` especially).

21 Accessibility

Add modal semantics (focus trap, Escape handling, distinct labels) to the mobile nav drawer. Add combobox semantics to search suggestions. Improve cart quantity button labels and dark-surface text contrast.

22 Performance

Introduce a caching strategy (route-level `revalidate` or `unstable_cache`) for catalog data that doesn't need to be fully dynamic on every request. Add `sizes` to the handful of `fill` images missing it. Investigate collapsing the duplicate per-request auth calls.

23 SEO

Set `NEXT_PUBLIC_SITE_URL` . Add canonical + hreflang alternates. Add per-page metadata to home/cart/search. Add Twitter Card tags. Add `Organization` / `WebSite` / `BreadcrumbList` JSON-LD. Disallow `/search` in robots.txt.

Missing functionality

- Password reset (Supabase Auth recovery flow is available but unwired on the frontend/route side).
- Proof-of-payment upload path (columns exist, no storage policy or upload code exists yet).
- Any test suite whatsoever — no framework, no config, no files.

Partial functionality

- Payment methods: RLS supports full CRUD, UI only supports Update.
- Banners: RLS supports full CRUD, UI has no edit-in-place.
- Box contents save: functionally complete but not transactional.

Bugs

- Payment-verification control has no error handling — RPC exceptions vanish silently.
- `adminSetBoxContents`'s delete-then-insert can leave a box empty if the insert half fails.
- `adminSetPrimaryProductImage`'s two sequential updates (clear old primary, set new) aren't atomic — a failure between them can leave a product with no primary image.

Security issues

- None CRITICAL/HIGH. Two MEDIUM items: leaked-password protection disabled in Auth settings; migration `0021` untracked in migration history.

Performance issues

- 110 instances of stacked permissive RLS policies on the same table/role/action (functionally fine, pure per-row overhead).
- 18 unused indexes (expected at current test-scale row counts, not yet actionable).
- No data caching anywhere — every request re-queries Supabase from zero.

Business decisions needed

- Subscription auto-payment/auto-redemption behavior on renewal.
- Reservation cutoff/lead-time policy for chicken products.
- Whether boolean availability is sufficient long-term or real stock-quantity tracking is needed.
- Refund/cancellation policy.

✓ What Is Done

The entire trusted-mutation architecture: every order, loyalty, subscription-status, and role change goes through a SECURITY DEFINER function with its own internal authorization check, live-verified with zero CRITICAL/HIGH findings across 23 tables. The full commerce loop — browse, search, cart, checkout with real slot-capacity locking and minimum-order/date/zone validation, order creation, status progression, payment verification, tracking, and reorder — works end-to-end against real data. The loyalty engine (earn/redeem/reverse/manual-adjust) reconciles exactly against its own ledger. The subscription renewal engine is a real, cron-scheduled, idempotent (hard unique-index-backed) background job, not a stub. Admin has full operational control over orders, staff/roles, delivery configuration, and loyalty. The bilingual AR/RTL & EN/LTR system and the storefront's glassmorphism design system are both genuinely, carefully built.

● What Is Partially Done

Green Box bundles, product images, and homepage banners are all fully *functional* mechanisms sitting on *zero real content* — every one of them is code-complete and data-empty. The CMS covers banners and one settings row, leaving most customer-facing marketing copy hardcoded in translation files. Several admin CRUD surfaces (banners, categories, payment methods, delivery areas) support less than the RLS layer actually allows. Error handling is inconsistent: some controls (order status, subscription actions) handle failure well; their near-identical siblings (payment verification, checkout, profile, address forms) don't.

● What Is Missing

Password reset, a customer-facing subscription detail page, any test coverage whatsoever, and real photography for all 68 products. Proof-of-payment upload has schema support but no code path. No caching strategy exists anywhere. No canonical/hreflang/Twitter SEO metadata exists.

⚠ What Is Broken

Checkout and subscription-creation discard the specific, already-computed server error and show a generic failure instead. Login doesn't honor `?next=`, stranding guests on the homepage after signing in. The admin payment-verification control fails completely silently on error. The "store name" CMS field saves data that is never displayed anywhere. Three CTAs show the wrong label due to a reused translation key. The product-page breadcrumb and the reorder button both have a direction bug.

What Needs UI/UX Work

Reconcile the admin dashboard's flat visual language with the storefront's glassmorphism — right now they read as two different products. Consolidate ad-hoc border-radius and box-shadow values into the existing token system. Add proper modal/combobox accessibility semantics to the mobile nav drawer and search suggestions. Add pagination to search results. Add real success/error feedback consistently across every admin edit form.

What Needs Backend Work

Make box-contents save and primary-image-setting atomic. Add real error surfacing to payment verification. Reconcile migration `0021`'s tracking gap. Enable leaked-password protection. Design a customer-scoped storage policy before building proof-of-payment upload. Purge test/QA data before go-live.

What Needs Business Decisions

Subscription auto-payment/redemption behavior on renewal. Chicken/reservation cutoff and lead-time policy. Whether the current boolean product-availability model needs to evolve into real stock-quantity tracking. Refund/cancellation policy. The correct real WhatsApp contact number.

What Must Be Fixed Before Launch

The checkout/subscription error-message bug, the login redirect bug, the silent payment-verification failure, setting `NEXT_PUBLIC_SITE_URL`, disallowing `/search` in robots.txt, building a password-reset flow, and uploading real product/category/banner photography. None of these are architecturally hard — all are contained, well-understood fixes or content tasks against an already-solid backend.

What Can Wait

Everything else in \$27's Medium/Low lists: box-content-save atomicity, banner/category edit-in-place, payment-method Create/Delete UI, reactivate buttons, dead-code removal, canonical/hreflang metadata, caching/ISR, the subscription detail page, and the broader admin visual redesign. None of these block a customer from successfully placing and receiving a real order today.

Recommended Next Implementation Order

1. **Fix the four launch-blocking bugs first** — checkout/subscription error surfacing, login `?next=`, payment-verification error handling, and the environment/robots.txt SEO fixes. All are small, contained, high-value changes to code that already exists and works otherwise.
2. **Content sprint:** real product photography (at least the top-selling/most-visible categories first), one real homepage banner, one real Green Box bundle. This unlocks three features that are currently fully built but invisible.
3. **Password reset flow** — the one genuinely missing piece of core authentication.

4. **Resolve the standing business decisions** (§27F) — none require code changes to identify, and several (reservation cutoff, refund policy) block confidently telling customers what to expect.
 5. **Testing foundation:** introduce at least integration tests around the trusted-function layer (order creation, loyalty, subscription renewal) before making further changes to it — this is the highest-leverage place to add tests given how much business logic already lives there correctly.
 6. **Admin visual consistency pass** and the remaining Medium/Low UX polish items, once the above is stable.
 7. **Data hygiene:** purge test/QA data and reconcile the migration-history gap immediately before go-live, not before — there's no reason to lose the ability to keep testing against it until the last moment.
-

Audit performed 2026-09-13 against the live repository at `C:\Users\akkrramm\OneDrive\Desktop\GREEN BOX` and its connected Supabase project. Methodology: four independent research passes (storefront frontend, admin dashboard, backend/database/security, cross-cutting SEO/performance/accessibility/visual) plus direct verification of `tsc`, `eslint`, `next build`, and targeted live-database queries. No code, configuration, or database state was modified during this audit.